

L. Balanced Unshuffle (Hard)

time limit per test: 2 seconds🕒

memory limit per test: 256 megabytes

The only difference with the medium version is the maximum length of the input.

A *parentheses sequence* is a string consisting of characters "(" and ")", for example "(() ()".

A *balanced parentheses sequence* is a parentheses sequence which can become a valid mathematical expression after inserting numbers and operations into it, for example "(() (()))".

The *balance* of a parentheses sequence is defined as the number of opening parentheses "(" minus the number of closing parentheses ")". For example, the balance of the sequence "(() (()" is 3.

A balanced parentheses sequence can also be defined as a parentheses sequence with balance 0 such that each of its prefixes has a non-negative balance.

We define the *balanced shuffle* operation that takes a parentheses sequence and returns a parentheses sequence as follows: first, for every character of the input sequence, we compute the balance of the prefix of the sequence *before* that character and write those down in a table together with the positions of the characters in the input sequence, for example:

Prefix balance	0	1	2	1	2	3	2	1
Position	1	2	3	4	5	6	7	8
Character	(	(	)	(	(	)	)	)

Then, we sort the columns of this table in *increasing* order of prefix balance, breaking ties in *decreasing* order of position. In the above example, we get:

Prefix balance	0	1	1	1	2	2	2	3
Position	1	8	4	2	7	5	3	6
Character	(	)	(	(	)	(	)	)

The last row of this table forms another parentheses sequence, in this case "() (() ())". This sequence is called the result of applying the balanced shuffle operation to the input sequence, or in short just the balanced shuffle of the input sequence.

Surprisingly, it turns out that the balanced shuffle of any balanced parentheses sequence is always another balanced parentheses sequence (we will omit the proof for brevity). Even more surprisingly, the balanced shuffles of two different balanced parentheses sequences are always different, therefore the balanced shuffle operation is a bijection on the set of balanced parentheses sequences of any given length (we will omit this proof, too).

You are given a balanced parentheses sequence. Find its preimage: the balanced parentheses sequence the balanced shuffle of which is equal to the given sequence.

Input

The only line of input contains a string s consisting only of characters "(" and ")". This string is guaranteed to be a non-empty balanced parentheses sequence with its **length not exceeding 500 000**.

Output

Print the balanced parentheses sequence t such that the balanced shuffle of t is equal to s . It is guaranteed that the answer always exists and is unique.

Example

input	Copy
()((()())	
output	Copy
((()((()())	